

Récapitulatif — Projet de Programmation ENSAE 2026

Cheminements de réflexion et observations expérimentales

Partie 1 — Chemin de temps minimal

1.1 — Graphe simple (fatigue ignorée)

On ignore la fatigue en construisant un graphe simple via `build_simple_graph`. On applique Dijkstra classique. Le coût d'une arête est simplement sa longueur.

1.2 — Graphe étendu

Le problème central est que le temps de parcours d'une arête dépend de la fatigue accumulée avant de l'emprunter — Dijkstra classique ne peut pas gérer des poids variables.

Solution : encoder la fatigue dans le nom du nœud.

- Un nœud devient un tuple (sommet, F) où F est la fatigue accumulée.
- Une arête $(u, F) \rightarrow (v, F+f)$ a un poids fixe $(1+F) \times \text{longueur}(u \rightarrow v)$.
- Dijkstra est applicable car les poids sont maintenant constants.

Bijection : chaque chemin dans le réseau correspond exactement à un chemin dans le graphe étendu avec le même coût. Minimiser l'un revient à minimiser l'autre.

$F_{\max} = (n-1) \times \max(\text{fatigues})$: borne sur la fatigue maximale d'un chemin simple (au plus $n-1$ arêtes). Les nœuds (u, F) avec $F > F_{\max}$ ne sont jamais atteints.

Nœud source : $(\text{start}, 0)$. **Condition d'arrêt** : $u_{\text{actuel}}[0] == \text{end}$ (on ignore la fatigue d'arrivée inconnue a priori).

1.3 — GraphImplicit

`build_extended_graph` crée tous les nœuds (u, F) pour tous les F de 0 à $F_{\max}$ — même ceux jamais atteints. Sur un grand réseau avec grandes fatigues, $F_{\max}$ explose.

Solution : GraphImplicit hérite de `Graph` et redéfinit uniquement la méthode de voisinage. Les voisins de (u, F) sont calculés à la demande au lieu d'être stockés.

	<code>build_extended_graph</code>	<code>GraphImplicit</code>
Nœuds créés	Tous les (u, F)	Seulement ceux visités
Arêtes stockées	Toutes	Aucune
Calcul des voisins	Lecture du dictionnaire	Calculé à la demande
Mémoire	$O(n \times F_{\max})$	$O(\text{nœuds visités})$

Adaptation de `shortest_path` : paramètre `is_target=None` pour gérer les deux cas. Sans heuristique : on compare le nœud directement à la cible. Avec graphe étendu : on ignore la composante fatigue et on ne compare que le sommet.

Partie 2 — Améliorations algorithmiques

2.1 — Pareto Pruning

Un nœud (v, F) est **Pareto dominé** si on a déjà atteint v avec un temps $t' \leq t$ ET une fatigue $F' \leq F$. Continuer depuis (v, F, t) ne peut que donner de moins bons résultats.

- Ne change pas le résultat optimal — seulement la vitesse.
- Implémenté via un dictionnaire pareto : sommet $\rightarrow$ liste de (temps, fatigue) non dominés.
- Paramètre optionnel `pruning=False` dans `shortest_path` pour rétrocompatibilité.
- Un compteur `compteur_prune` mesure les nœuds éliminés.

2.2 — Algorithme A*

Dijkstra explore dans toutes les directions (trie par $g(u)$ = distance depuis source). A* oriente l'exploration vers la cible en triant par $f(u) = g(u) + h(u)$ où $h(u)$ est une estimation du temps restant. Dijkstra est un cas particulier de A* avec $h = 0$.

Admissibilité : h doit toujours sous-estimer le vrai coût restant.

- Si h surestime $\rightarrow$ on risque de rater le chemin optimal.
- Si h sous-estime $\rightarrow$ optimal garanti mais on explore plus de nœuds.
- h parfaite = vrai coût restant $\rightarrow$ aucun nœud inutile exploré.

Deux heuristiques ont été implémentées :

Heuristique 1 — sans fatigue : $h(u, F) =$ distance minimale de u à end dans le graphe simple (fatigue ignorée, $F=0$). Admissible car $(1+0) \leq (1+F)$ pour tout $F \geq 0$. Précalculée en une seule passe par Dijkstra à rebours depuis end sur le graphe inversé.

Heuristique 2 — avec fatigue fixe : $h(u, F) =$ distance minimale de u à end avec la fatigue fixée à la valeur F avec laquelle on arrive en u (elle n'augmente pas sur le chemin restant). Admissible car une fatigue fixe sous-estime la vraie fatigue croissante. Meilleure que $h=0$ car elle intègre l'état actuel du joueur, mais nécessite un Dijkstra par valeur de F — coûteux en précalcul. C'est le meilleur compromis qualité/coût accessible sans oracle.

2.3 — Résultats expérimentaux sur `medium-smallfatigue.txt`

Les quatre combinaisons testées donnent toutes une distance optimale identique de **29 934**, confirmant que le pruning et A* n'affectent pas la correction du résultat, uniquement la vitesse. Le pruning réduit le temps d'un facteur ~ 6 (de 0,0440 s à 0,0073 s), tandis que A* seul apporte peu de gain sur ce fichier — voire légèrement plus lent que Dijkstra, ce qui s'explique par le surcoût du calcul de l'heuristique sur un graphe de taille modérée.

Combinaison	Distance	Temps	Nœuds prunés	Nœuds explorés
Dijkstra seul (pruning=False, h=0)	29 934	0,0440 s	—	—
Pruning seul (pruning=True, h=0)	29 934	0,0073 s	982	—
A* seul (pruning=False, h>0)	29 934	0,0506 s	—	—
A* + Pruning (pruning=True, h>0)	29 934	0,0078 s	969	—

Sur **large-largefatigue.txt** : sans pruning → timeout ($F_{\max}$ trop grand, graphe étendu énorme).
Avec pruning → faisable. Montre que le pruning est indispensable sur les grands graphes.

2.4 — Résultats expérimentaux sur large-smallfatigue.txt

Sur ce fichier de grande taille, les combinaisons sans pruning sont en timeout — le graphe étendu est trop volumineux pour être exploré sans élagage. Le pruning devient indispensable. A* apporte ici un gain significatif (~17 % de réduction du temps), ce qui s'explique par la taille du graphe : l'heuristique oriente efficacement l'exploration et compense largement son surcoût de calcul.

Combinaison	Distance	Temps	Nœuds prunés
Dijkstra seul (pruning=False, h=0)	—	timeout	—
Pruning seul (pruning=True, h=0)	2 000 993	21,4233 s	558 048
A* seul (pruning=False, h>0)	—	timeout	—
A* + Pruning (pruning=True, h>0)	2 000 993	17,6870 s	410 710

build_extended_graph vs GraphImplicit : la construction de build_extended_graph est incluse dans le temps mesuré — sur les grands fichiers la différence est très significative.

Partie 3 — Extension multi-missions

Modèle retenu

Un agent enchaîne plusieurs missions dans l'ordre. La fatigue s'accumule sur toute la séquence : mission 1 + trajet inter-mission + mission 2 + ...

- La fatigue n'est pas remise à zéro entre les missions.
- Le trajet entre v_{1t} et v_{2s} contribue aussi à la fatigue.

Schéma complet :

Départ : ($v1s$, $F=0$)	→ Mission 1 →	Arrivée ($v1t$, $F1$)
	→ Trajet inter-mission →	Arrivée ($v2s$, $F2$)
	→ Mission 2 →	Arrivée ($v2t$, $F3$)

Problème de l'approche naïve

Enchaîner trois appels indépendants à `shortest_path` optimise chaque mission localement sans tenir compte de l'impact de la fatigue sur la suivante.

Exemple :

- Chemin A pour mission 1 : temps=10, fatigue finale=100
- Chemin B pour mission 1 : temps=12, fatigue finale=5

`shortest_path` choisit A car $10 < 12$, mais la mission 2 sera bien plus lente à cause de la fatigue 100. B était globalement meilleur !

Solution retenue — Pareto multi-missions

Au lieu de garder seulement le chemin de temps minimal vers v_{1t} , on garde tous les chemins **Pareto-optimaux** — tous les couples (temps, fatigue) tels qu'aucun autre chemin n'est meilleur sur les deux critères à la fois.

Exemple :

- Chemin A : temps=10, fatigue=100 → non dominé
- Chemin B : temps=12, fatigue=5 → non dominé
- Chemin C : temps=11, fatigue=80 → dominé par A sur temps ET par B sur fatigue → éliminé

On teste A et B comme points de départ pour la mission suivante et on prend le meilleur temps total. C'est le Pareto pruning réutilisé, mais pour garder plusieurs options ouvertes plutôt que pour accélérer.